§ COMP 11104 · AGILE CLOUD AUTOMATION · GROUP COURSEWORK

The Flexible Product Catalogue *Sprint 1 Increment*

REPOSITORY

github.com/Emrebener/ACA_Coursework

Group Members

DEVELOPER · SCRUM MASTER

Emre Bener

@Emrebener

DEVELOPER · PRODUCT OWNER

Zubair Khalid

@zubairKhalid-UK

DEVELOPER

Moiz Ahmad

@Moizkhanhoh

DEVELOPER

Muhammad Arslan

@arslan437

DEVELOPER

Muhammad Sohail

@muhammadsohailmian24-spec

Product Backlog

Sprint goal: deliver a containerised catalogue, with `dynamic faceted filtering` over MongoDB, deployed continuously to an HTTPS test environment.

Items are labelled functional or non-functional, sized with T-shirts, and prioritised with MoSCoW on the GitHub Project board.

PBI	ITEM	MOSCOW
01–05	CRUD for flexible products	MUST · in
06–08	Category filter & dynamic facets	MUST · in
10, 14	Docker & Compose deployment	MUST · in
11	Vagrant dev VM	MUST · in
13	Jenkins declarative pipeline	MUST · in
17	Increment demo recording	MUST · in
12	One-command test runner	SHOULD · in
09	Search by name / description	SHOULD · out
15	Sort by price / date	COULD · out
16	Optional product image URL	COULD · out

Dynamic Facet Filtering

Each product stores a category-specific `attributes` object in MongoDB. The app inspects what is in the database and generates the filter UI, with no hard-coded schema per category.

SCREENSHOT · HEADPHONES

app.3-10-97-37.nip.io/products?category=Headphones → facets: brand · noiseCancelling · connection · colour

SCREENSHOT · LAPTOPS

app.3-10-97-37.nip.io/products?category=Laptops → facets: brand · cpu · ramGb · storageGb · ports

CRUD with Flexible Attributes

ATTRIBUTES JSON · HEADPHONES JSON

```
{
  "brand": "Bose",
  "noiseCancelling": true,
  "batteryLifeHours": 24
}
```

SCREENSHOT · NEW PRODUCT FORM

Name: Bose QC Ultra · Price: 379.99 · Condition: New · Category: Headphones

01 Create: backend stores attributes as-is.

02 View: detail page renders flexible attributes.

03 Edit: price 379.99 to 349.99.

04 Filter: appears under `noiseCancelling=true` instantly.

05 Delete: confirm, back to a clean seed.

Faceted filtering

Letting a user narrow a list by multiple attribute filters at once, for example `brand = Sony`, `noiseCancelling = true`, `connection includes Bluetooth`.

The facets are the filter controls themselves: the sidebar on any e-commerce site.

Dynamic facets

The available filters depend on the data, not a predefined schema. A laptop listing shows RAM and CPU filters; headphones show noise cancellation and battery life; books show author and genre.

Different sets of facets for different slices of the same collection. That is the hard part of this feature.

EXAMPLE DOCUMENT · HEADPHONES

DB.PRODUCTS.FINDONE() JSON

```json
{
  "name": "Sony WH-1000XM5",
  "category": "Headphones",
  "price": 329.00,
  "condition": "New",
  "attributes": {
    "brand": "Sony",
    "noiseCancelling": true,
    "connection": "Bluetooth",
    "batteryLifeHours": 30,
    "colour": "Black"
  }
}
```

Common fields up top; the `attributes` sub-document holds whatever keys make sense for the category.

Pipeline-as-Code

STAGE 01
Checkout
GitHub webhook on
push to develop

→

STAGE 02
Install
npm ci inside the
Node 22 (alpine) container

→

STAGE 03
Run tests
jest --runInBand
9 unit tests · green

→

STAGE 04
Build image
Single Dockerfile,
tagged per build

→

STAGE 05
Deploy
Docker Compose up
on AWS EC2 test env

TRIGGER
GitHub webhook → HMAC-signed → Jenkins

TRANSPORT
Caddy + Let's Encrypt · TLS termination

CODE
Declarative `Jenkinsfile` in SCM

SCREENSHOT · JENKINS CONSOLE

aca-coursework → latest green build → Pipeline Steps / Console Output

Development Infrastructure

The same `docker-compose.yml` brings up the app and MongoDB stack on a Vagrant Ubuntu VM for development, and on AWS EC2 via Jenkins for the continuously-deployed test environment.

`Vagrantfile`

Ubuntu VM · libvirt provider · one-command up.

`scripts/provision-vagrant.sh`

Installs Docker, Node.js, mongosh, lazydocker, and project dependencies on every fresh VM.

`Dockerfile`

Node 22 (alpine) base image. Shared by local and test-env builds.

`docker-compose.yml`

App + MongoDB + named volume. Identical service definitions everywhere.

`Caddyfile`

TLS termination via Let's Encrypt ACME. nip.io wildcard DNS. Fronts both Jenkins and the app on separate hostnames.

`Jenkinsfile`

Declarative pipeline, committed to SCM, not configured in the UI.

Testing Strategy

We used Test-Driven Development where practical, writing tests ahead of their corresponding modules. Jest runs on every Jenkins build via the `Run tests` stage. One manual test case exercises the full create-filter-delete loop against the live environment.

`productValidation.test.js`

- Required-field rules (name, category, price).
- Price coercion from string to Number.
- Attribute JSON parsing & rejection.

`facetService.test.js`

- Dynamic facet derivation per category.
- Array-value flattening (e.g. `ports`).
- MongoDB filter construction.

SCREENSHOT · JEST IN TERMINAL

npm test output · productValidation + facetService, green run

SCREENSHOT · JENKINS RUN TESTS STAGE

Pipeline view · Run tests step passing on latest build

§ INCREMENT · SPRINT 1 · COMP11104

Thank you.

app.3-10-97-37.nip.io · test environment